



UNIVERSITY OF PISA
MASTER'S DEGREE IN CYBERSECURITY

LIGHT HASH V5
COURSE HARDWARE AND EMBEDDED SECURITY

Author(s)
Francesco Camaccioli
Matteo Giannini

Academic year 2023/2024

Contents

1	Project Specifications	1
1.1	Introduction	1
1.2	Specification Analysis	1
1.2.1	Light Hash V5 Algorithm	1
1.2.2	SA - State Array Function	2
1.2.3	Θ - Theta Function	2
1.2.4	ρ - Rho Function	2
1.2.5	FPX - Final permutation and XORing Function	2
2	High-level Model	3
2.1	Introduction	3
2.2	Code Analysis	3
2.2.1	light_hash_v5	3
2.2.2	SA	4
2.2.3	Theta	4
2.2.4	Rho	4
2.2.5	FPX	4
2.3	Code Testing and Output	5
2.3.1	Testing	5
2.3.2	Output	5
3	RTL Design	6
3.1	Project Structure	6
3.2	Base modules	7
3.2.1	SA.sv	7
3.2.2	Theta.sv	8
3.2.3	Rho.sv	8
3.2.4	FPX.sv	8
3.2.5	aux_conv.sv	9
3.3	hash_round.sv	9
3.4	hash_block.sv	10

3.5	light_hash_top.sv	13
3.6	Architecture Block Diagram	16
3.6.1	Hash Round Block Diagram	16
3.6.2	36-Round Hashing Block Diagram	16
4	Interface Specifications and Expected Behavior	18
4.1	Inputs and Outputs	18
4.2	How It Works	19
4.2.1	Input Accumulation	19
4.2.2	Processing the Block	19
4.2.3	Completion Signal	19
4.3	Waveforms	19
5	Functional Verification	23
5.1	Introduction	23
5.2	Test Case 1	24
5.3	Test Case 2 & 3	25
5.4	Test Case 4	25
5.5	Test 5 - Corner Case	26
6	FPGA Implementation Results	28
6.1	Flow Summary and Resource Usage	28
6.2	Static Timing Analysis	30

CHAPTER 1

Project Specifications

1.1 Introduction

The following project aims at developing Light Hash v5, a Hashing algorithm that receives a 64-bit input data block and generates a 64-bit output digest.

In this Chapter we analyze the algorithm's layout and its inner functions.

1.2 Specification Analysis

1.2.1 Light Hash V5 Algorithm

The main part the algorithm is the following 36 round loop, that calls four utility functions: **SA** (1.2.2), Θ (1.2.3), ρ (1.2.4) and **FPX** (1.2.5), all deeply analyzed below.

```
1 for (r = 0; r < 36; r++) {  
2     if (r == 0) {  
3         H = SA(m);  
4     }  
5     else {  
6         H = SA(H);  
7     }  
8     H =  $\Theta$ (H);  
9     H =  $\rho$ (H);  
10 }  
11 d = FPX(H);
```

In which:

- r is the current round number

- m is the message
- H is the 8-byte output digest during computation
- d is the final output digest

1.2.2 SA - State Array Function

Function that transforms the 64-bit input message into an 8-bytes array, XORing each message byte with the corresponding byte of the Initialization Vector.

```
1 for (i = 0; i < 8; i++) {
2     H[i] = m[i] XOR IV[i];
3 }
```

In which IV is the fixed Initialization Vector of 8-bytes with initial values:

$IV[7] = [0x34, 0x55, 0x0F, 0x14, 0xCC, 0xAA, 0xF0, 0xE3]$

This function is used in the first round with the message as input, and in the other remaining rounds the input is instead the hash calculated in the previous round stored in H .

1.2.3 Θ - Theta Function

Permutation function that basically inverts the input array.

```
1 for (i = 0; i < 8; i++) {
2     H[i] = H[7-i];
3 }
```

1.2.4 ρ - Rho Function

Function that performs an hexadecimal sum over each element of the input array with a fixed value, then a modular reduction with another fixed hexadecimal value over the result of the previous operation.

```
1 for (i = 0; i < 8; i++) {
2     H[i] = (H[i] + 0x85) mod 0xFD;
3 }
```

1.2.5 FPX - Final permutation and XORing Function

Final Permutation function that generates the output digest by XOR-ing the input with the Initialization Vector, starting from the right-most element of the input.

```
1 for (i = 0; i < 8; i++) {
2     d[i] = H[7-i] XOR IV[i]
3 }
```

This function is used just once, at the end of the rounds computation, and it takes the last round's output as input, generating the final output digest.

CHAPTER 2

High-level Model

2.1 Introduction

This chapter analyzes the high-level model of the hashing algorithm, developed in Python. The aim of this model is to have a software implementation that will be used to check the correctness of the RTL Model. The test vector used in this model is the same as the first test vector of the RTL implementation, contained in the *modelsim/tv* directory. The script also calculates the execution time of the algorithm, by subtracting the time the main function call from the time of the main function return.

2.2 Code Analysis

In this section we will analyze each Python function, starting from the main one and then analyzing each sub-function call, in the following subsections.

2.2.1 light_hash_v5

Similarly to the specification, this function takes in input the 64-bit message block, instantiates H, the auxiliary digest for the computation and runs the 36 round algorithm, as shown before in the chapter 1. At the end of the function, it performs the final permutation and xoring function and returns its 64-bit output digest.

```
1 def light_hash_v5(m):  
2     H = []  
3     # Perform 36 rounds of the hashing process  
4     for r in range(36):  
5         if(r == 0):  
6             H = SA(m)
```

```

7         else:
8             H = SA(H)
9             H = Theta(H)
10            H = Rho(H)
11
12            # Final Permutation and Xoring
13            digest = FPX(H)
14            # Returning final digest
15            return digest

```

2.2.2 SA

The State Array (SA) function Transforms the 64-bit input data block by XOR-ing it with the corresponding Initialization Vector element, bitwise.

```

1 def SA(m) :
2     for i in range(8):
3         # XOR with the corresponding IV element
4         m[i] ^= IV[i]
5     return m

```

2.2.3 Theta

The Theta function takes the input 8-byte array and reverses it.

```

1 def Theta(H) :
2     H.reverse() # reverse the array in place
3     return H # return the reversed array

```

2.2.4 Rho

The Rho function Adds 0x85 to each element of the input the array, mod 0xFD.

```

1 def Rho(H) :
2     for i in range(8):
3         H[i] = (H[i] + 0x85) % 0xFD
4     return H

```

2.2.5 FPX

The Final Permutation and Xoring function computes the XOR on the input 8-byte array with the Initialization Vector in reverse order.

```

1 def FPX(H) :
2     d = [0] * 8
3     for i in range(8):
4         d[i] = H[7 - i] ^ IV[i] # XOR with IV in reverse
5         order
6     return d

```

2.3 Code Testing and Output

2.3.1 Testing

In the Python script, the algorithm is tested with the input test vector "a1b2c3d4e5f60788", which is the same value as the first test vector for the RTL implementation, contained in *modelsim/tv/test1.hex*.

The following lines of code also contain the check on the input message length, if this is shorter than 8 bytes, the message is split in blocks of 8 bytes, and a padding of 0x00 is added at the beginning of the last block, if needed.

```
1 message = bytes.fromhex("A1B2C3D4E5F60788")
2
3 block_size = 8 #64 bits in bytes
4
5 if len(message) % block_size != 0:
6     padding = block_size - (len(message) % block_size)
7     message += b'\x00' * padding
8     print(f"Padding_added:_{padding}_bytes")
9
10 for i in range(0, len(message), block_size):
11     block = bytearray(8)
12     for i in range(8):
13         block[i] = (int.from_bytes(message, byteorder='
14         big', signed=False) >> (8 * (7 - i))) & 0xFF
15     print("Input_Block:", '_'.join(f'0x{byte:02X}' for
16         byte in block))
17
18     start = time.time()
19     digest = light_hash_v5(block) # calculate the digest
20     of the block
21     end = time.time()
22
23     print("Output_Digest:", '_'.join(f'0x{byte:02X}' for
24         byte in digest))
25 print(f"Execution_time:_{(end-_start)_*_1e9:.0f}_
26     nanoseconds")
```

2.3.2 Output

Running the script returns the following output, which we expect to be the same as the output of the RTL implementation.

```
macOS:~/Desktop/HES/light_hash_v5 % py model/light_hash_v5_high_level.py
Input Block: 0xA1 0xB2 0xC3 0xD4 0xE5 0xF6 0x07 0x88
Output Digest: 0xBC 0x9A 0x01 0x2E 0xF8 0x35 0xF5 0x02
Execution time: 88930 nanoseconds
```

CHAPTER 3

RTL Design

In this chapter we will analyze the overall structure of the RTL project, starting from a brief overview of each file and then describing them deeply in dedicated sections. We chose to have a bottom-up view of the project, starting from the *Base Modules* and then analyzing how these are used inside the following Sections.

3.1 Project Structure

Inside the *db* folder there are the following System Verilog files:

- **hash_block.sv:**
This module coordinates the functionality of the hash algorithm, over a single block of 64-bits. It computes the hash output using the *hash_round* and *FPX* modules.
- **hash_round.sv:**
This module implements the core round logic of the hash function. It contains the operations performed during each iteration of the hash algorithm, its role is fundamental because it handles the input and outputs of each sub-module.
- **light_hash_top.sv:**
This is the top-level module of the project and it integrates the hash processor of (*hash_block*) into a more complete system, handling message blocks and coordinating the start and end of processing of each one of them.
- **aux_conv.sv:**
This module provides auxiliary conversion from an 8-bytes array to a full 64-bit byte string.

- **SA.sv:**

This module implements the State Array function through a combinational logic that performs the XOR operation of each byte of the input message, with the Initialization Vector, which is contained into a *reg* variable.

- **theta.sv:**

This module implements the Theta function through a combinational logic that performs the inversion of the input byte-array.

- **rho.sv:**

This module implements the Rho function through a combinational logic that performs the sum of each element of the input message byte-array with 0x85, mod 0xFD. The operation is performed inside an *always_comb* block, making this module purely combinational.

- **FPX.sv:**

This module implements the Final Permutation and Xor-ing function by performing the XOR of the input message in reverse order with the Initialization Vector, inside an *always_comb* block.

3.2 Base modules

In this section we show the code of every base module, all working as specified in 1.

3.2.1 SA.sv

```
1 module SA (
2     input  [63:0] msg_block,
3     output reg [7:0] output_block [0:7]
4 );
5     reg [7:0] IV [0:7];
6     initial begin
7         IV[0] = 8'h34;
8         IV[1] = 8'h55;
9         IV[2] = 8'h0F;
10        IV[3] = 8'h14;
11        IV[4] = 8'hCC;
12        IV[5] = 8'hAA;
13        IV[6] = 8'hF0;
14        IV[7] = 8'hE3;
15    end
16
17    always_comb begin
18        output_block[0] = IV[0] ^ msg_block[63:56];
19        output_block[1] = IV[1] ^ msg_block[55:48];
20        output_block[2] = IV[2] ^ msg_block[47:40];
21        output_block[3] = IV[3] ^ msg_block[39:32];
22        output_block[4] = IV[4] ^ msg_block[31:24];
```

```

23         output_block[5] = IV[5] ^ msg_block[23:16];
24         output_block[6] = IV[6] ^ msg_block[15:8];
25         output_block[7] = IV[7] ^ msg_block[7:0];
26     end
27 endmodule

```

3.2.2 Theta.sv

```

1 module theta(
2     input wire [7:0] msg_arr [0:7],
3     output reg [7:0] output_arr [0:7]
4 );
5
6     always_comb begin
7         for (int i = 0; i < 8; i = i + 1) begin
8             output_arr[i] = msg_arr[7-i];
9         end
10    end
11 endmodule

```

3.2.3 Rho.sv

```

1 module rho(
2     input wire [7:0] msg_arr [0:7],
3     output reg [7:0] output_arr [0:7]
4 );
5     always_comb begin
6         for (int i = 0; i < 8; i = i + 1) begin
7             output_arr[i] = (msg_arr[i] + 9'h85) % 9'hFD;
8         end
9     end
10 endmodule

```

3.2.4 FPX.sv

```

1 module FPX(
2     input wire [7:0] msg_arr [0:7],
3     output reg [7:0] output_arr [0:7]
4 );
5
6     reg [7:0] IV [0:7];
7     initial begin
8         IV[0] = 8'h34;
9         IV[1] = 8'h55;
10        IV[2] = 8'h0F;

```

```

11         IV[3] = 8'h14;
12         IV[4] = 8'hCC;
13         IV[5] = 8'hAA;
14         IV[6] = 8'hF0;
15         IV[7] = 8'hE3;
16     end
17
18     always_comb begin
19         for (int i = 0; i < 8; i = i + 1) begin
20             output_arr[i] <= IV[i] ^ msg_arr[7-i];
21         end
22     end
23 endmodule

```

3.2.5 aux_conv.sv

```

1  module aux_conv(
2      input wire  [7:0] msg_arr [0:7],
3      output reg  [63:0] linear_arr
4  );
5      integer i;
6      always_comb begin
7          linear_arr = {msg_arr[0], msg_arr[1], msg_arr[2],
8                      msg_arr[3], msg_arr[4], msg_arr[5], msg_arr
9                      [6], msg_arr[7]};
10     end
11 endmodule

```

3.3 hash_round.sv

This module implements the single round of the hashing algorithm by:

1. receiving the input 64-bits message.
2. performing SA over the input message, storing the output inside *SA_r*.
3. calculating Theta over *SA_r*, storing the output inside *theta_r*.
4. computing Rho over *theta_r*, storing the output in *rho_r*.
5. providing the output as an array of 8 bytes, inside *output_block*, and as a linear byte-string, inside *linear_out*.

```

1 module hash_round(
2     input wire  [63:0] msg_block,
3     output reg  [7:0] output_block [0:7],
4     output reg  [63:0] linear_out
5 );

```

```

6
7     reg [7:0] SA_r [0:7];
8     reg [7:0] theta_r [0:7];
9     reg [7:0] rho_r [0:7];
10    reg [63:0] aux_conv_r;
11
12    SA SA_inst (
13        .msg_block(msg_block),
14        .output_block(SA_r)
15    );
16
17    theta theta_inst (
18        .msg_arr(SA_r),
19        .output_arr(theta_r)
20    );
21
22    rho rho_inst (
23        .msg_arr(theta_r),
24        .output_arr(rho_r)
25    );
26
27    aux_conv aux_conv_inst (
28        .msg_arr(rho_r),
29        .linear_arr(aux_conv_r)
30    );
31
32    assign output_block = rho_r;
33    assign linear_out = aux_conv_r;
34
35 endmodule

```

3.4 hash_block.sv

The `hash_top` module serves as controller for the hash function over a single message block. It coordinates the 36 rounds of the *hash_round* module through a *Finite State Machine* (FSM) and controls the flow of data. Let us now examine all the components managed by this module:

- **msg_block**: 64-bits block to be hashed
- **ctxt**: 64-bit output digest
- **Control Signals**:
 - **dav_**: (Data Valid) active-low signal that notifies if input data block is available on the `msg_block` variable, triggers new input processing cycle.

- `rfd`: (Ready For Data) active-high signal that notifies if output digest is ready to be read from outside modules, and that the module is ready to receive another input data block.
- `rst_n`: active-low signal that resets the FSM to its initial state.

- **Internal Registers:**

- `state`: A 2-bit register holding the current state of the FSM, which controls the flow of the module.
- `round_buffer`: buffer that holds intermediate hash values between rounds.
- `round_count`: counter to keep track of the current round being processed.

- **Finite State Machine**

The FSM has three possible states, shown graphically in the following Diagram (3.1):

1. IDLE

- The FSM waits for the `dav_` signal to indicate that input data is available.
- When data is available (`dav_ == 0`), it initializes the `round_buffer` with the input message block and resets the `round_count`.
- The FSM transitions to the `PROCESS` state to start processing the hash rounds.

2. PROCESS

- The FSM processes the hash rounds.
- It updates the `round_buffer` with the output of the current hash round (`hash_round_out`).
- The `round_count` is incremented with each round.
- When `round_count` reaches 35, the FSM transitions to the `FINALIZE` state.

3. FINALIZE

- The FSM transfers the final hash output from `fpx_output_arr`, which is the output of the `FPX` module instance, to the `ctxt` array.
- It displays a message indicating the completion of the processing round.
- The FSM waits for the `dav_` signal to indicate that new data is available before transitioning back to the `IDLE` state.

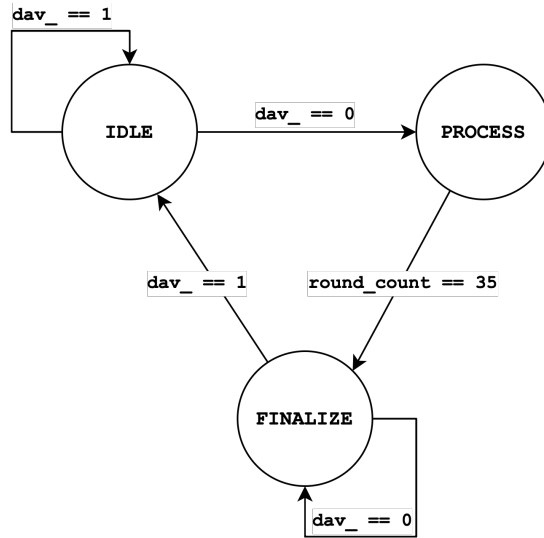


Figure 3.1: FSM Diagram of `hash_block.sv` (3.4)

```

1 always_ff @(posedge clk or negedge rst_n) begin
2     if (!rst_n) begin
3         state <= IDLE;
4         round_buffer <= 64'd0;
5         round_count <= 6'd0;
6         RFD <= 1;
7     end else begin
8         case (state)
9             IDLE: begin
10                RFD <= (dav_==0)? 0 : 1;
11                round_buffer <= msg_block;
12                round_count <= 6'd0;
13                state <= (dav_==0)? PROCESS : IDLE;
14            end
15            PROCESS: begin
16                RFD <= 0;
17                if (round_count < 6'd35) begin
18                    round_buffer <= hash_round_out;
19                    round_count <= round_count + 1;
20                end else begin
21                    state <= FINALIZE;
22                end
23            end
24            FINALIZE: begin
25                for(int i = 0; i < 8; i = i + 1) begin
26                    txt[i] <= fpx_output_arr[i];
27                end
28                display("PROCESS:_Round_%d_completed",

```

```

29         round_count);
30         RFD <= 0;
31         state <= (dav_==1)? IDLE : FINALIZE;
32     end
33 endcase
34 end
end

```

3.5 light_hash_top.sv

This module's role is to receive messages and split them into 64-bits input block, to pass them as input to an instance of the hash_processor module of previous section (3.4). This module uses a *Finite State Machine* aswell to control the processing of message block and to handle its task.

Let us now have a look at the core components of this module:

- **Control Signals:**

- byte_valid: Active-high signal indicating that the current byte on msg_byte is valid and can be processed.
- processing_n: Active-high signal indicating whether a block is currently being processed.
- eoc: The End of Conversion signal, which goes high when the hashing process is complete, signaling that the output digest is ready to be read.
- rst_n: Active-low reset signal that re-initializes the module's *FSM* to its IDLE state when low.

- **Internal Registers:**

- state: A 2-bit register holding the current state of the *FSM*, which controls the flow of the module.
- byte_count: A 3-bit counter that tracks the number of bytes accumulated in the current block. It helps in determining when the block is ready for processing.
- eoc_reg: End of Conversion signal, which goes high when the hashing process is complete, signaling that the output digest is ready to be read.

- **Finite State Machine**

This module's *FSM* has four possible states, as shown in the figure below(3.2):

1. IDLE

- The *FSM* waits for the input message to begin.
- As bytes arrive, they are accumulated into the 64-bit msg_block register.
- When the block is full (i.e., byte_count == 7) or when the message ends (msg_start == 0), the *FSM* transitions to the PROCESS state.

2. PROCESS

- In this state, the module waits for the `hash_processor` to process the accumulated block.
- The `processing_n` signal is cleared, signaling to the `hash_processor` that the block is ready.
- The output of the `hash_processor` is copied into the `ctxt` array to be read from outside, then the state is updated to `DONE` to start the resetting phase.

3. DONE

- The `eoc_reg` is set high to signal that the output digest is ready.
- The `msg_block` is cleared, and the *FSM* resets the byte counter, preparing to transition back to the `IDLE` state for a new message block.

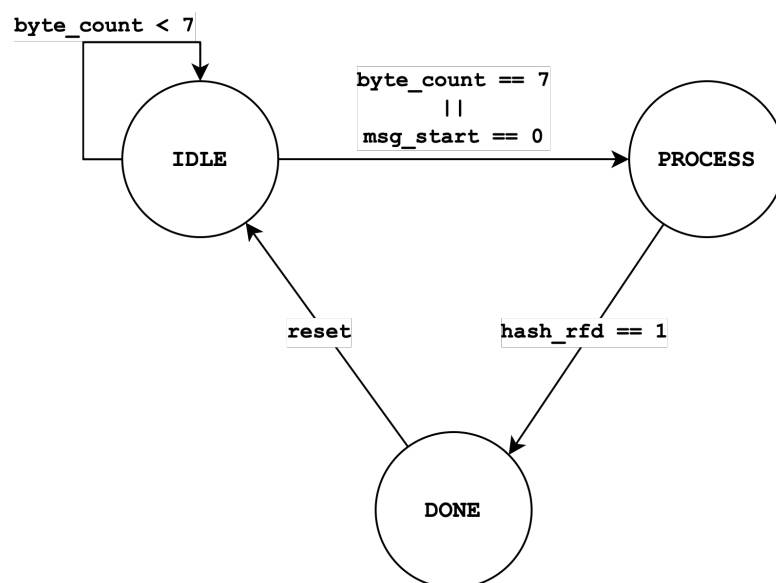


Figure 3.2: FSM Diagram of `light_hash_top.sv` (3.5)

```
1 always_ff @(posedge clk or negedge rst_n) begin
2     if (!rst_n) begin
3         state <= IDLE;
4         msg_block <= 64'd0;
5         byte_count <= 3'd0;
6         processing_n <= 1'b1;
7         eoc_reg <= 1'b1;
8     end else begin
9         case (state)
10            IDLE: begin
11                eoc_reg <= 1'b1;
```

```

12         if (msg_start && byte_valid && byte_count
13             <=7) begin
14             msg_block <= {msg_block[55:0],
15                 msg_byte};
16             byte_count <= (byte_count == 3'd7)?
17                 byte_count:byte_count + 1;
18             state <= IDLE;
19         end
20     else if (byte_count == 7 (!msg_start &&
21         byte_count>0))begin
22         processing_n <= 1'b0;
23         state <= (hash_rfd==0) ? PROCESS :
24             IDLE;
25         eoc_reg <= (hash_rfd==0) ? 1'b0:1'b1;
26     end
27 else begin
28     msg_block <= msg_block;
29     byte_count <= byte_count;
30     state <= IDLE;
31 end
32 end
33 PROCESS: begin
34     processing_n <= 1'b1;
35     eoc_reg <= 1'b0;
36     if (hash_rfd==1) begin
37         for(int i = 0; i < 8; i = i + 1)
38             begin
39                 ctxt[i] <= hash_output[i];
40             end
41         state <= DONE;
42     end
43 end
44 DONE: begin
45     eoc_reg <= 1'b1;
46     msg_block <= 64'd0;
47     byte_count <= 3'd0;
48     state <= IDLE;
49 end
50 endcase
51 end

```

3.6 Architecture Block Diagram

In this section we want to show the Block Diagrams of the developed module, representing both the round function functionality and how it is used by the upper layer in the 36-rounds hashing.

3.6.1 Hash Round Block Diagram

In Figure 3.3 it is possible to see the logic interconnection of the sub-modules of the hashing algorithm, outlining the order in which the functions operate on the input message block.

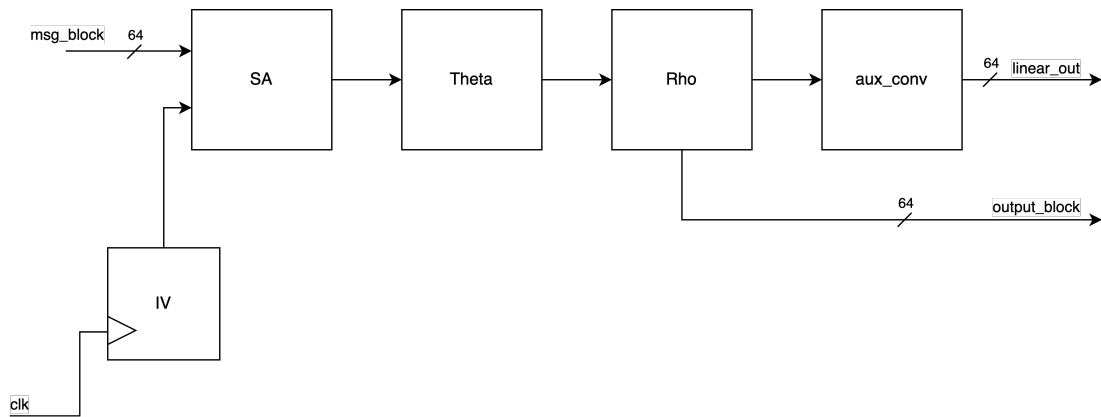


Figure 3.3: Hash Round Block Diagram

3.6.2 36-Round Hashing Block Diagram

In Figure 3.4 we display the previous diagram of figure 3.3 as a single module called "hash_round". It is also possible to follow how the output of each round is saved inside the round_buffer register and then used as input for the next computation, until the check over the round_count register done via a XOR with the decimal value 35 triggers the demultiplexer that outputs the final digest to the last sub-module, the Final Permutation and XORing function.

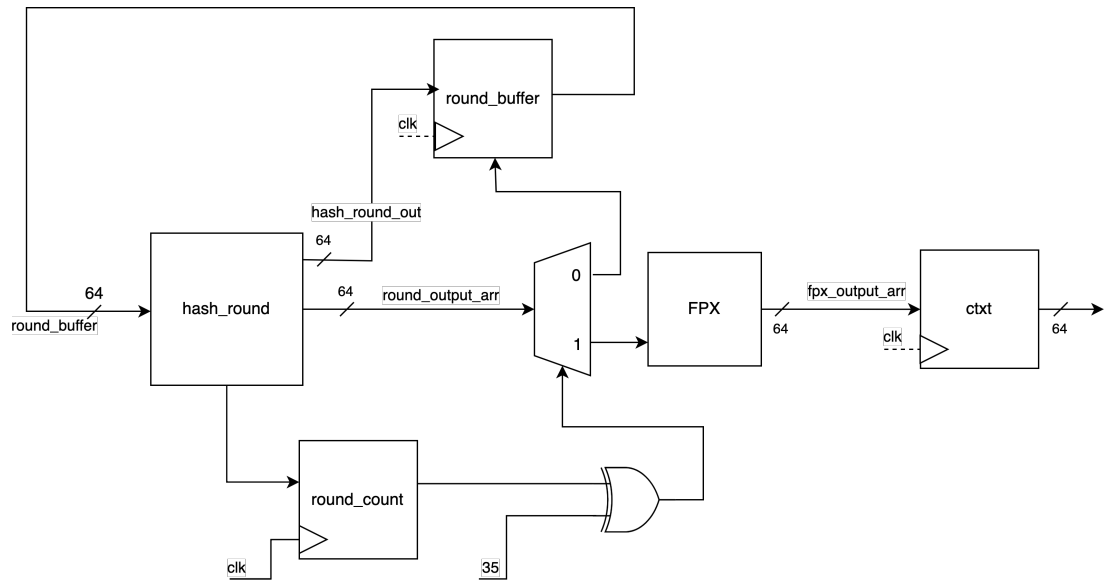


Figure 3.4: 36-Round Hashing Block Diagram

CHAPTER 4

Interface Specifications and Expected Behavior

This chapter is dedicated to the instruction for the input/output ports, how they should be used inside practical uses of the module and finally we will show the waveforms of the Test Cases they we performed. The code of the test cases is explained in the next chapter - *Functional Verification* (5).

4.1 Inputs and Outputs

Inputs:

- `clk`: Clock signal.
- `rst_n`: Active-low reset signal to initialize the module.
- `msg_byte`: 8-bit input data, representing a byte of the message being hashed.
- `byte_valid`: A signal that indicates when the `msg_byte` is valid and should be read.
- `msg_start`: A signal that indicates when the message is being provided. This should be put at 1 when the message starts and reverted back to 0 when the message ends. The module also supports incomplete messages with less than 64 bits.

Outputs:

- `eoc`: End-of-conversion signal that is put to 1 when the computation ends and the digest is ready.
- `ctxt`: Array representing the output digest. This is provided as an array of eight 8-bits elements.

4.2 How It Works

4.2.1 Input Accumulation

The module accumulates up to 8 bytes of data which needs to be provide one at the time. The input byte is provided through `msg_byte` and is valid when `byte_valid` is high. The accumulation continues until the user provides 8 bytes or until the user puts `msg_start` to 0. If the user wishes to provide more than 8 bytes as input, they should wait until the previous 8 bytes are processed and keep `msg_start` to 1. In case of an incomplete message a padding of zeros is added starting from the MSB.

4.2.2 Processing the Block

Once a block of 8 bytes has been fully accumulated, or when the input message is incomplete but finished, the `light_hash` module signals the internal `hash_processor` to begin processing by putting `processing_n` to 0. This register is in fact connected to the `dav_` (Data Valid) input of the `hash_processor` to signal that the input data are valid and to start the computation. The `hash_processor` performs a series of hash rounds and outputs the result as an 8-byte digest. When the hash is ready, the `hash_processor` signals the completion of the computation to the top level model by setting its `rfd` (Ready for Data) output to 1.

4.2.3 Completion Signal

When the processing is complete, the `eof` signal goes high, indicating that the output digest is ready. The result can be read from the `ctxt` output, which contains eight 8-bit values representing the final hash digest.

4.3 Waveforms

In the figure (4.1) we show the starting of the simulation, with the `msg_byte` wave changing from its uninitialized value of all "0x00" hexes to the input of the first Test Case (5.1), "a1 b2 c3 d4 e5 f6 07 88". The `byte_valid` and `msg_start` waves are both correctly set to high, and the `eof` wave is set to low after 20ps from the end of the reading phase, signaling that the computation has started. In the figure (4.2) we can follow the end of computation signal until it gets updated to high, signaling that the computation has ended and the output is ready to be read.

We can in fact see the output being copied into `ctxt`, which is updated to "bc 9a 01 2e f8 35 f5 02". In figure (4.2) we can also see the input of the Test Case 2 getting read, while the output of the previous computation is returned.

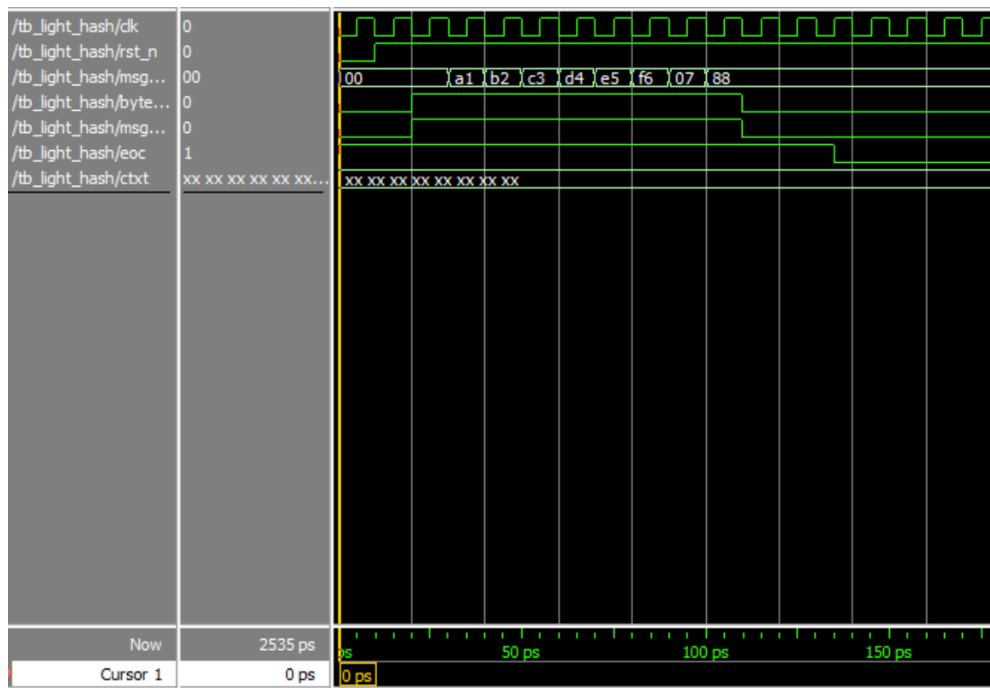


Figure 4.1: Test Case 1 Input waveform

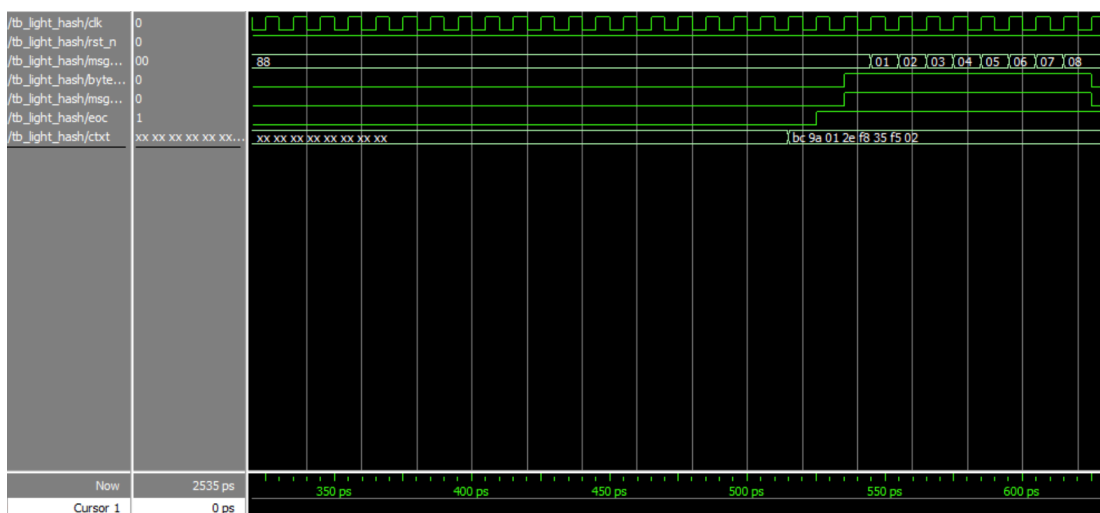


Figure 4.2: Test Case 1 Output & Test Case 2 Input

In the figure (4.3) it is possible to observe the output of the Test Case 2 and the input of full "0x00" hexes, of the third test. At right-most part of the picture we can also see the output of the Test Case 3, and the input of the following one, that will be zoomed in on the picture below.

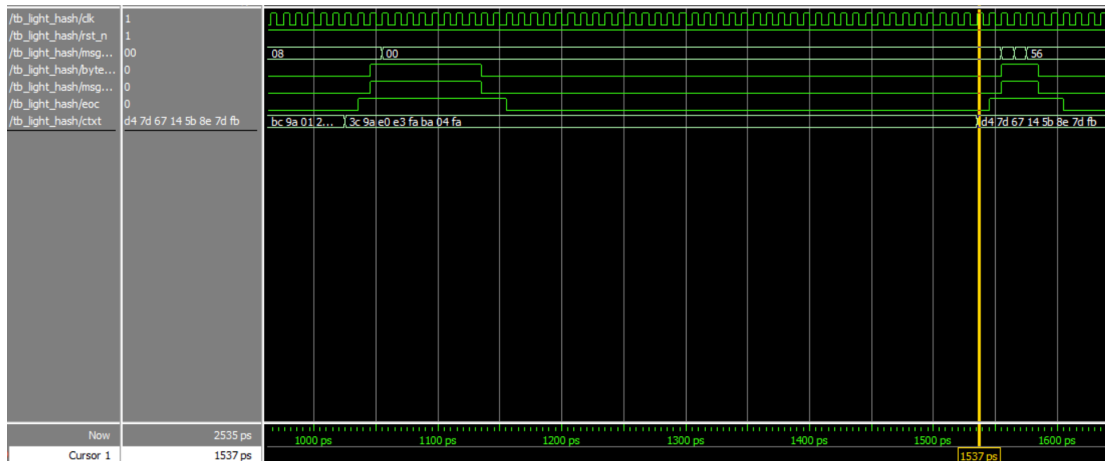


Figure 4.3: Test Case 2 Output & Test Case 3 Input

In the next Figure (4.4) we can follow the `byte_valid` rapidly changing during the reading of the input message and the `msg_start` working as expected. Even in this case, the output gets generated and saved into the `ctxt`, with the value "e0, e6, 14, 72, 5b, 8e, 7d, fb".

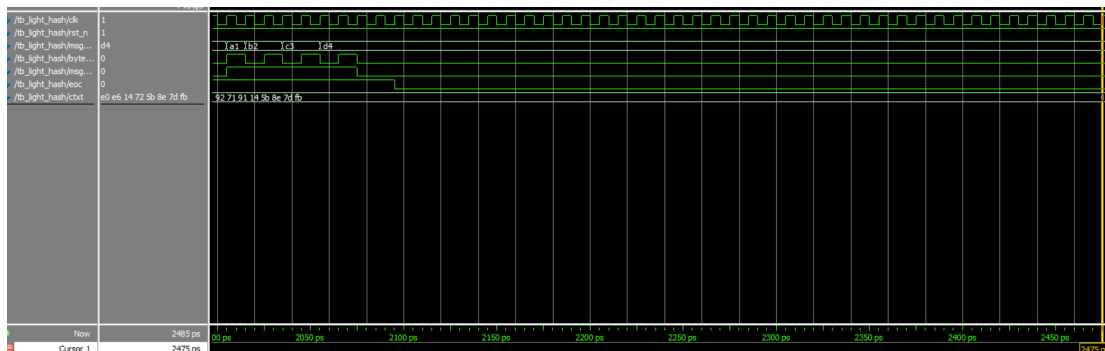


Figure 4.4: Test Case 4 Input and Output

Zooming in into the Test Case 4 Input (4.5), we can follow the `byte_valid` signal going low just after the third byte read from the input, meaning that the input is not a full 8-byte sized message. In the next Figure (4.6), we can check the final part, that works as expected, with the end of conversion signal going high right 10ps after the copying of the output over `ctxt`, signaling that the output can be read.

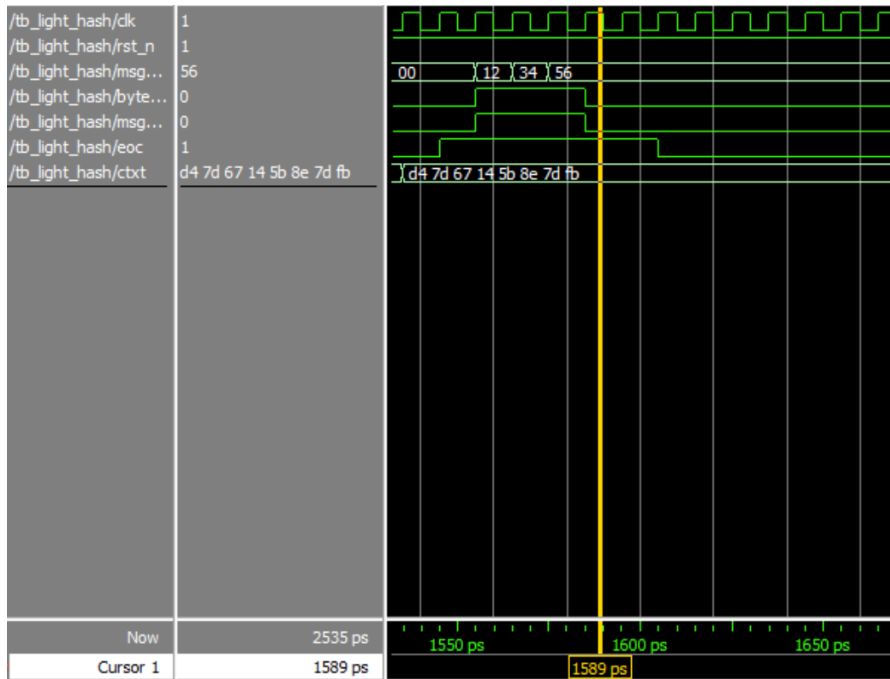


Figure 4.5: Test 5 Input

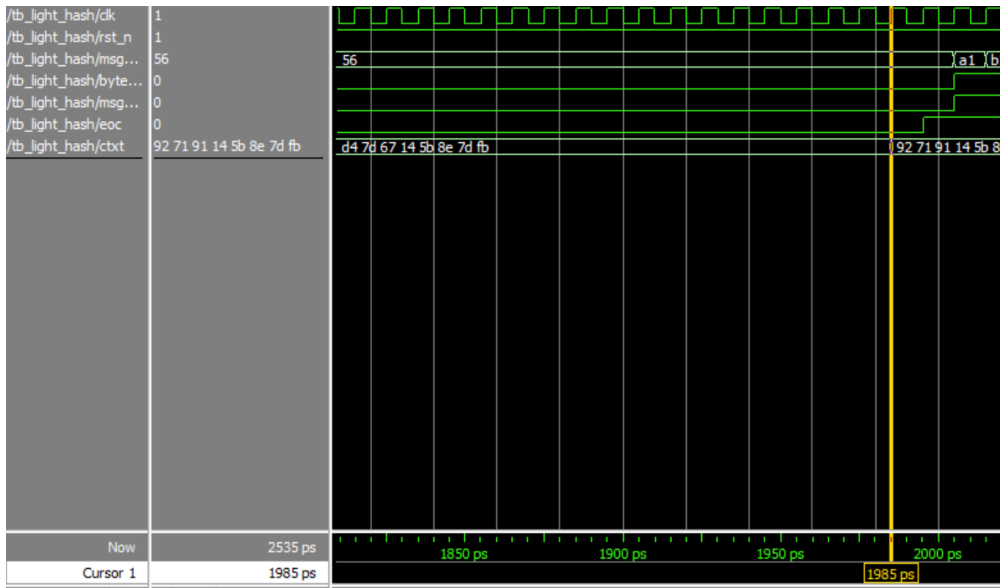


Figure 4.6: Test Case 5 Output

CHAPTER 5

Functional Verification

5.1 Introduction

This chapter is dedicated to the Verification and Testing on the developed module, performed through the *tb/light_hash_tb.sv* file. Using Modelsim it is possible to simulate the execution of the model over the developed testcase, obtaining results comparable with the Chapter 2's results. Since they both have the same final output digest over the Test Case 1 (5.1), we can conclude that both the high level model and the RTL one are performing the same exact operations.

Let's now analyze how the testbench unit is developed, starting from the instantiation of the *light_hash* module and the creation of the clock signal:

```
1 module tb_light_hash;
2     reg clk;
3     reg rst_n;
4     reg [7:0] msg_byte;
5     reg byte_valid
6     reg msg_start;
7     wire eoc;
8     wire [7:0] ctxt [0:7];
9
10    light_hash uut (
11        .clk(clk),
12        .rst_n(rst_n),
13        .msg_byte(msg_byte),
14        .byte_valid(byte_valid),
```

```

15         .msg_start(msg_start),
16         .eoc(eoc),
17         .ctxt(ctxt)
18     );
19     always #5 clk = ~clk;

```

Over this *light_hash module* instantiation we performed **5** different tests, shown in the following subsections:

5.2 Test Case 1

The Test Case 1 sets the `msg_start` and `byte_valid` to high to signal the start of the input validity, then reads the content of the Test Vector 1, contained in the *modelsim/tv* folder, and passes it, cycling byte per byte, to the instantiation of the *light_hash module*, through the `msg_byte` register. At the end of the 8-byte read, the `msg_start` and `byte_valid` are updated to low, to signal the end of the validity of the input. The testbench then waits for the end-of-conversion signal to go high, before outputting the final digest.

```

42 // Test Case 1
43 $display("----- Test Case 1 -----");
44
45 msg_start = 1;
46 byte_valid = 1;
47
48 // reading from test vector file 1
49 $readmemh("../modelsim/tv/test1.hex", msg_bytes);
50 #10;
51 // putting the bytes into the hash input byte
52 for (int i = 0; i < 8; i++) begin
53     msg_byte = msg_bytes[i];
54     $display("Processing msg_byte = %h", msg_byte);
55     #10;
56 end
57
58 // updating the message start and byte valid signals
59 msg_start = 0;
60 byte_valid = 0;
61
62 wait(eoc == 0);
63
64 // waiting for the end of conversion signal to go high
65 wait(eoc == 1);
66
67 // printing output digest
68 $display("Output Digest: %h %h %h %h %h %h %h %h",
69     ctxt[0], ctxt[1], ctxt[2], ctxt[3], ctxt[4], ctxt[5], ctxt[6], ctxt[7]);
70
71 #10;

```

Figure 5.1: "Test Case 1"

Here we continue with the content of the Test Vector 1 file (*modelsim/tv/test1.hex*):

```
1 a1 b2 c3 d4 e5 f6 07 88
```

The output is the following, which is the same as the Python implementation (1):

```
1 bc 9a 01 2e f8 35 f5 02
```

5.3 Test Case 2 & 3

The Test Cases 2 and 3, similarly to the first one, use the *test2.hex* and *test3.hex* files from the same folder as before, with the following inputs:

Test Case 2 Input:

```
1 01 02 03 04 05 06 07 08
```

Test Case 3 Input:

```
1 00 00 00 00 00 00 00 00
```

Test Case 2 Output:

```
1 3c 9a e0 e3 fa ba 04 fa
```

Test Case 3 Output:

```
1 d4 7d 67 14 5b 8e 7d fb
```

5.4 Test Case 4

In this Test Case we use a 4-bytes input in which every input byte is provided by putting the *byte_valid* signal to 0 between every byte. Even in this case the input is read correctly and the output corresponds to the expected one.

```

// Message of 4 bytes with msg_start high and byte_valid low between bytes
$display("----- Test Case 4 -----");

// Send 4 bytes sequentially
msg_start = 1;
byte_valid = 1;
msg_byte = 8'hA1; #10;
byte_valid = 0;
msg_byte = 8'hB2; #10;
byte_valid = 1;
#10
byte_valid = 0;
msg_byte = 8'hC3; #10;
byte_valid = 1;
#10
byte_valid = 0;
msg_byte = 8'hD4; #10;
byte_valid = 1;
#10

// Complete the message
msg_start = 0;
byte_valid = 0;

wait(eoc == 0);

// Wait for processing
wait(eoc == 1);
$display("Output ctxt = {%h, %h, %h, %h, %h, %h, %h, %h}",
ctxt[0], ctxt[1], ctxt[2], ctxt[3], ctxt[4], ctxt[5], ctxt[6], ctxt[7]);

```

Figure 5.2: "Test Case 4"

The output is the following:

```
1 e0, e6, 14, 72, 5b, 8e, 7d, fb
```

5.5 Test 5 - Corner Case

The Test Case 5 works slightly differently in respect to the previous ones, since it gives an input of only 3 bytes to the light_hash module. The purpose Corner Case is to test the handling of shorter input-messages through padding. The input message is not read from a file as before, but this time it is manually set to the desired byte value with a pause of 10ps in between single bytes, to give the time to the model to read the buffer before going to the next one.

```

// Test Case 5
$display("----- Test Case 5 -----");

// Send 3 bytes message
msg_start = 1;
byte_valid = 1;
msg_byte = 8'h12; #10;
msg_byte = 8'h34; #10;
msg_byte = 8'h56; #10;

// Signaling that the message is complete
msg_start = 0;
byte_valid = 0;

wait(eoc == 0);

// Wait for processing
wait(eoc == 1);
$display("Output ctxt = {%h, %h, %h, %h, %h, %h, %h, %h}",
ctxt[0], ctxt[1], ctxt[2], ctxt[3], ctxt[4], ctxt[5], ctxt[6], ctxt[7]);

```

Figure 5.3: "Test Case 5"

The output is the following:

```

1 92, 71, 91, 14, 5b, 8e, 7d, fb

```

CHAPTER 6

FPGA Implementation Results

In the final chapter of this report we will analyze the results of the Quartus implementation of the RTL model over the FPGA device "*Intel - FPGA Cyclone® V*", with name "*5CGXFC9D6F27C7*".

We used Virtual Pins over the module, with exception to the clock signal, which is kept as physical signal.

6.1 Flow Summary and Resource Usage

This section is dedicated to the Flow summary of the Quartus Synthesis, show in the Figure 6.1, that includes the overall resource usage of the FPGA board, the number of physical and virtual pins. In the Figure 6.2 we can see the usage of the ALMs, both the ones used for the virtual pins' connection and the ones dedicated to the FPGA's logic and registers. The total pins row signals the single physical pin used, which is, as we said before, the clock signal.

Flow Status	Successful - Thu Oct 24 12:08:17 2024
Quartus Prime Version	20.1.1 Build 720 11/11/2020 SJ Lite Edition
Revision Name	hash_top
Top-level Entity Name	light_hash
Family	Cyclone V
Device	5CGXFC9D6F27C7
Timing Models	Final
Logic utilization (in ALMs)	283 / 113,560 (< 1 %)
Total registers	294
Total pins	1 / 378 (< 1 %)
Total virtual pins	76
Total block memory bits	0 / 12,492,800 (0 %)
Total DSP Blocks	0 / 342 (0 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	0 / 17 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 6.1: Flow Summary

	Resource	Usage	%
1	Logic utilization (ALM... total ALMs on device)	283 / 113,560	< 1 %
2	$\vee$ ALMs needed [=A-B+C]	283	
1	$\vee$ [A] ALMs used in fin...placement [=a+b+c+d]	274 / 113,560	< 1 %
1	[a] ALMs used for LUT logic and registers	100	
2	[b] ALMs used for LUT logic	136	
3	[c] ALMs used for registers	38	
4	[d] ALMs used for me... half of total ALMs)	0	
2	[B] Estimate of ALMs...ble by dense packing	29 / 113,560	< 1 %
3	$\vee$ [C] Estimate of ALMs...available [=a+b+c+d]	38 / 113,560	< 1 %
1	[a] Due to location constrained logic	0	
2	[b] Due to LAB-wide signal conflicts	0	
3	[c] Due to LAB input limits	0	
4	[d] Due to virtual I/Os	38	
3			
4	Difficulty packing design	Low	
5			
6	$\vee$ Total LABs: partially or completely used	33 / 11,356	< 1 %
1	-- Logic LABs	33	
2	-- Memory LABs (up to half of total LABs)	0	
7			
8	$\vee$ Combinational ALUT usage for logic	450	
1	-- 7 input functions	0	
2	-- 6 input functions	70	
3	-- 5 input functions	37	
4	-- 4 input functions	30	
5	-- ≤ 3 input functions	313	
9	Combinational ALUT usage for route-throughs	55	
10			
11	$\vee$ Dedicated logic registers	294	
1	$\vee$ -- By type:		

Figure 6.2: Resource Allocation

6.2 Static Timing Analysis

This section is dedicated to the last task of the Quartus Synthesis, the Timing Analysis. In the folder *quartus/constr* we have the *constr.sdc* file that defines the timing constraints for the model. The file's content is shown in the figure 6.3. The clock period is set to 8.76 nanoseconds, resulting in a frequency of approximately 114.16 MHz, which is the lowest possible frequency that satisfies the timing requirements.

```
1  set CLK_PERIOD_NS 8.76
2  set MIN_IO_DELAY [expr double($CLK_PERIOD_NS)/10.0]
3  set MAX_IO_DELAY [expr double($CLK_PERIOD_NS)/5.0]
4  create_clock -name clk -period $CLK_PERIOD_NS [get_ports clk]
5  set_false_path -from [get_ports rst_n] -to [get_clocks clk]
6  set_input_delay -min $MIN_IO_DELAY [all_inputs] -clock [get_clocks clk]
7  set_input_delay -max $MAX_IO_DELAY [all_inputs] -clock [get_clocks clk]
8  set_output_delay -min $MIN_IO_DELAY [all_outputs] -clock [get_clocks clk]
9  set_output_delay -max $MAX_IO_DELAY [all_outputs] -clock [get_clocks clk]
```

Figure 6.3: Timing Constraints

In the figures below we have the results of the Static Timing Analysis phase, most importantly during the worst-case scenarios, which are the *Slow 1100 mv 85°C* and *0°C* analysis cases. The requirements are met in both of them, but the overall worst case is the 85°C test, with a maximum frequency of 116.54 MHz.

Slow 1100mV 0C Model Fmax Summary				
<<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	116.54 MHz	116.54 MHz	clk	

Figure 6.4: Slow 1100mv 85C Model Summary

Slow 1100mV 85C Model Fmax Summary				
<<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	123.43 MHz	123.43 MHz	clk	

Figure 6.5: Slow 1100mv 0C Model Summary